OSLAB

Skill-14

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

void execute_command(char *input)
{
   char *args[MAX_ARGS];
   int argc = 0;

   char *append_file = NULL;
   char *error_file = NULL;

   char *token = strtok(input, " \t\n");

   while (token != NULL)
   {
      // ==============================
      // APPEND REDIRECTION >>
      // ==============================
      if (strcmp(token, ">>") == 0)
      {
         token = strtok(NULL, " \t\n");

         if (token == NULL)
         {
            printf("Error: missing append file\n");
            return;
```

```
            }

            append_file = token;
        }

        // ==============================
        // STDERR REDIRECTION 2>
        // ==============================
        else if (strcmp(token, "2>") == 0)
        {
            token = strtok(NULL, " \t\n");

            if (token == NULL)
            {
                printf("Error: missing error output file\n");
                return;
            }

            error_file = token;
        }

        // Normal argument
        else
        {
            args[argc++] = token;
        }

        token = strtok(NULL, " \t\n");
    }

    args[argc] = NULL;

    if (argc == 0)
    {
        return;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }
```

```
if (pid == 0)
{
   // ==============================
   // APPEND OUTPUT >>
   // ==============================
   if (append_file != NULL)
   {
      int fd = open(
         append_file,
         O_WRONLY | O_CREAT | O_APPEND,
         0644
      );

      if (fd == -1)
      {
         perror("append file");
         exit(EXIT_FAILURE);
      }

      // Redirect stdout to file
      if (dup2(fd, STDOUT_FILENO) == -1)
      {
         perror("dup2 stdout");
         close(fd);
         exit(EXIT_FAILURE);
      }

      close(fd);
   }

   // ==============================
   // STDERR REDIRECTION 2>
   // ==============================
   if (error_file != NULL)
   {
      int fd = open(
         error_file,
         O_WRONLY | O_CREAT | O_TRUNC,
         0644
      );

      if (fd == -1)
      {
```

```
            perror("error file");
            exit(EXIT_FAILURE);
        }

        // Redirect stderr to file
        if (dup2(fd, STDERR_FILENO) == -1)
        {
            perror("dup2 stderr");
            close(fd);
            exit(EXIT_FAILURE);
        }

        close(fd);
    }

    // Execute command
    execvp(args[0], args);

    // This error goes to stderr
    perror("execvp");
    exit(EXIT_FAILURE);
  }

  // Parent waits
  waitpid(pid, NULL, 0);
}

int main()
{
  char input[MAX_INPUT];

  while (1)
  {
    printf("redirection$ ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
      break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
```

```
    {
        continue;
    }

    if (strcmp(input, "exit") == 0)
    {
        break;
    }

    execute_command(input);
  }

  return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~$ cd ~/OSLab
lahari@DESKTOP-AIC33F7:~/OSLab$ nano redirection.c
lahari@DESKTOP-AIC33F7:~/OSLab$ gcc redirection.c -o redirection
lahari@DESKTOP-AIC33F7:~/OSLab$ ./redirection
redirection$ echo Hello >> data.txt
redirection$ echo World >> data.txt
redirection$ exit
lahari@DESKTOP-AIC33F7:~/OSLab$ cat data.txt
Hello
World
lahari@DESKTOP-AIC33F7:~/OSLab$ ./redirection
redirection$ echo Third >> data.txt
redirection$ exit
lahari@DESKTOP-AIC33F7:~/OSLab$ cat data.txt
Hello
World
Third
lahari@DESKTOP-AIC33F7:~/OSLab$ ./redirection
redirection$ ls abc.txt 2> error.txt
redirection$ exit
lahari@DESKTOP-AIC33F7:~/OSLab$ cat error.txt
ls: cannot access 'abc.txt': No such file or directory
lahari@DESKTOP-AIC33F7:~/OSLab$ ./redirection
redirection$ ls 2> error.txt
Lab    data.txt   fork       mini_shell        named_createpipe     rd                 section13write    signal1
Test   error.txt  input.txt  mini_shell.c      output.txt           redirection        section13write.c  signal1.c
cow    file1.txt  lab.txt    minishell         pipes                redirection.c      shell_lab         signals
cow.c  file_io    memory     minishell.c       producer_consumer    section13_read.c   signal            skill4
cr     files.txt  memory.c   minishell.c.save  producer_consumer.c  section13pipescr.c signal.c          wr
redirection$ cat error.txt
redirection$ exit
lahari@DESKTOP-AIC33F7:~/OSLab$
```